

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 13 LECTURE 2

MEMORY

MANAGEMENT

WISH YOU ALL THE BEST

❑ LECTURE OUTLINE

❖ Dynamic Memory Allocation

❖ Techniques for handling physically contiguous memory areas :

- Page Frame Management
- Memory Area Management

❖ Technique for handling non-contiguous memory areas

- Non-contiguous Memory Area Management

Page Frame Management

- ★ Linux adopts the smaller 4 KB page frame size as the standard memory allocation unit.
- ★ This makes things simpler for two reasons:
- ★ In the first case, the Page Fault exceptions issued by the paging circuitry are easily interpreted.
- ★ Either the page requested exists but the process is not allowed to address it, or the page does not exist.
- ★ In the second case, the memory allocator must find a free 4 KB page frame and assign it to the process.
- ★ Although both 4 KB and 4 MB are multiples of all disk block sizes, transfers of data between main memory and disks are in most cases more efficient when the smaller size is used.

Page Frame Management



❑ Page Descriptors

- The kernel must keep track of the current status of each page frame.
- For instance, it must be able to distinguish the page frames that are used to contain pages that belong to processes from those that contain kernel code or kernel data structures.
- Similarly, it must be able to determine whether a page frame in dynamic memory is free.
- A page frame in dynamic memory is free if it does not contain any useful data.
- It is not free when the page frame contains data of a User Mode process, data of a software cache, dynamically allocated kernel data structures, buffered data of a device driver, code of a kernel module, and so on.
- Essentially, the application may require that when a message is inserted in an empty queue, either a signal is sent to a selected process, or a new thread is created.



Page Frame Management



□ Page Descriptors

- State information of a page frame is kept in a page descriptor of type `page`, whose fields are shown in Table.
- All page descriptors are stored in the `mem_map` array.
- Because each descriptor is 32 bytes long, the space required by `mem_map` is slightly less than 1% of the whole RAM.
- The `virt_to_page(addr)` macro yields the address of the page descriptor associated with the linear address `addr`.
- The `pfn_to_page(pfn)` macro yields the address of the page descriptor associated with the page frame having number `pfn`.



Page Frame Management



❑ Page Descriptors

Table - The fields of the page descriptor

Type	Name	Description
unsigned long	flags	Array of flags Also encodes the zone number to which the page frame belongs.
atomic_t	_count	Page frame's reference counter.
atomic_t	_mapcount	Number of Page Table entries that refer to the page frame (-1 if none).
unsigned long	private	Available to the kernel component that is using the page (for instance, it is a buffer head pointer in case of buffer page; If the page is free, this field is used by the buddy system
Struct address_space *	mapping	Used when the page is inserted into the page cache, or when it belongs to an anonymous region
unsigned long	index	Used by several kernel components with different meanings. For instance, it identifies the position of the data stored in the page frame within the page's disk image or within an anonymous region or it stores a swapped-out page identifier
struct list_head	lru	Contains pointers to the least recently used doubly linked list of pages.

_count

A usage reference counter for the page. If it is set to -1, the corresponding page frame is free and can be assigned to any process or to the kernel itself. If it is set to a value greater than or equal to 0, the page frame is assigned to one or more processes or is used to store some kernel data structures. The page_count() function returns the value of the _count field increased by one, that is, the number of users of the page.

flags

Includes up to 32 flags that describe the status of the page frame. For each PG_xyz flag, the kernel defines some macros that manipulate its value. Usually, the PageXYZ macro returns the value of the flag, while the SetPageXYZ and ClearPageXYZ macro set and clear the corresponding bit, respectively.

Page Frame Management

❑ Page Descriptors

Table - Flags describing the status of a page frame

Flag name	Meaning
PG_locked	The page is locked; for instance, it is involved in a disk I/O operation.
PG_error	An I/O error occurred while transferring the page.
PG_referenced	The page has been recently accessed.
PG_uptodate	This flag is set after completing a read operation, unless a disk I/O error happened.
PG_dirty	The page has been modified
PG_lru	The page is in the active or inactive page list
PG_active	The page is in the active page list
PG_slab	The page frame is included in a slab
PG_highmem	The page frame belongs to the ZONE_HIGHMEM
PG_checked	Used by some filesystems such as Ext2 and Ext3
PG_arch_1	Not used on the 80 x 86 architecture.
PG_reserved	The page frame is reserved for kernel code or is unusable.
PG_private	The private field of the page descriptor stores meaningful data.
PG_writeback	The page is being written to disk by means of the writepage method
PG_nosave	Used for system suspend/resume.
PG_compound	The page frame is handled through the extended paging mechanism
PG_swapcache	The page belongs to the swap cache
PG_mappedtodisk	All data in the page frame corresponds to blocks allocated on disk.
PG_reclaim	The page has been marked to be written to disk in order to reclaim memory.
PG_nosave_free	Used for system suspend/resume.

Page Frame Management

❑ Non-Uniform Memory Access (NUMA)

- Linux supports the Non-Uniform Memory Access (NUMA) model, in which the access times for different memory locations from a given CPU may vary.
- The physical memory of the system is partitioned in several nodes.
- The time needed by a given CPU to access pages within a single node is the same.
- However, this time might not be the same for two different CPUs.
- For every CPU, the kernel tries to minimize the number of accesses to costly nodes by carefully selecting where the kernel data structures that are most often referenced by the CPU are stored.
- Furthermore, the Linux kernel makes use of NUMA even for some peculiar uniprocessor systems that have huge "holes" in the physical address space.
- The kernel handles these architectures by assigning the contiguous subranges of valid physical addresses to different memory nodes .

Page Frame Management



❑ Non-Uniform Memory Access (NUMA)

- The physical memory inside each node can be split into several zones. **Each node has a descriptor of type `pg_data_t`**, whose fields are shown in Table. All node descriptors are stored in a singly linked list, whose first element is pointed to by the `pgdat_list` variable.

The fields of the node descriptor

Type	Name	Description
struct zone []	node_zones	Array of zone descriptors of the node
struct zonelist []	node_zonelists	Array of zonelist data structures used by the page allocator
Int	nr_zones	Number of zones in the node
struct page *	node_mem_map	Array of page descriptors of the node
Struct bootmem_data *	bdata	Used in the kernel initialization phase
unsigned long	node_start_pfn	Index of the first page frame in the node
unsigned long	node_present_pages	Size of the memory node, excluding holes (in page frames)
unsigned long	node_spanned_pages	Size of the node, including holes (in page frames)
Int	node_id	Identifier of the node
pg_data_t *	pgdat_next	Next item in the memory node list
wait_queue_head_t	kswapd_wait	Wait queue for the kswapd pageout daemon
struct task_struct *	kswapd	Pointer to the process descriptor of the kswapd kernel thread
Int	kswapd_max_order	Logarithmic size of free blocks to be created by kswapd

Linux makes use of a single node that includes all system physical memory. Thus, the `pgdat_list` variable points to a list consisting of a single element the node 0 descriptor stored in the `contig_page_data` variable.

Page Frame Management

❑ Memory Zones

- In an ideal computer architecture, a page frame is a memory storage unit that can be used for anything: storing kernel and user data, buffering disk data, and so on.
- Every kind of page of data can be stored in a page frame, without limitations.
- However, real computer architectures have hardware constraints that may limit the way page frames can be used. In particular, the Linux kernel must deal with two hardware constraints of the 80 x 86 architecture:
 - The Direct Memory Access (DMA) processors for old ISA buses have a strong limitation: they are able to address only the first 16 MB of RAM.
 - In modern 32-bit computers with lots of RAM, the CPU cannot directly access all physical memory because the linear address space is too small.

Page Frame Management



❑ Memory Zones

- To cope with the two limitations due to hardware constraints, Linux partitions the physical memory of every memory node into three zones.
- In the 80 x 86 UMA architecture the zones are:
 - **ZONE_DMA** Contains page frames of memory below 16 MB
 - **ZONE_NORMAL** Contains page frames of memory at and above 16 MB and below 896 MB
 - **ZONE_HIGHMEM** Contains page frames of memory at and above 896 MB
- The **ZONE_DMA** zone includes page frames that can be used by old ISA-based devices by means of the DMA.
- The **ZONE_DMA** and **ZONE_NORMAL** zones include the "normal" page frames that can be directly accessed by the kernel through the linear mapping in the fourth gigabyte of the linear address space.
- Conversely, the **ZONE_HIGHMEM** zone includes page frames that cannot be directly accessed by the kernel through the linear mapping in the fourth gigabyte of linear address space.
- The **ZONE_HIGHMEM** zone is always empty on 64-bit architectures.

Page Frame Management

❑ Memory Zones

- Each memory zone has its own **descriptor of type zone**. Its fields are shown in Table.

Table- The fields of the zone descriptor

<u>Type</u>	<u>Name</u>	<u>Description</u>
unsigned long	free_pages	Number of free pages in the zone.
unsigned long	pages_min	Number of reserved pages of the zone
unsigned long	pages_low	Low watermark for page frame reclaiming; also used by the zone allocator as a threshold value
unsigned long	pages_high	High watermark for page frame reclaiming; also used by the zone allocator as a threshold value.
unsigned long []	lowmem_reserve	Specifies how many page frames in each zone must be reserved for handling low-on-memory critical situations.
struct per_cpu_pageset[]	pageset	Data structure used to implement special caches of single page frames
spinlock_t	lock	Spin lock protecting the descriptor.
struct free_area []	free_area	Identifies the blocks of free page frames in the zone
spinlock_t	lru_lock	Spin lock for the active and inactive lists.
struct list head	active_list	List of active pages in the zone
struct list head	inactive_list	List of inactive pages in the zone
unsigned long	nr_scan_active	Number of active pages to be scanned when reclaiming memory.
unsigned long	nr_scan_inactive	Number of inactive pages to be scanned when reclaiming memory

Page Frame Management

❑ Memory Zones

- Each memory zone has its own **descriptor of type zone**. Its fields are shown in Table.

Table- The fields of the zone descriptor

Type	Name	Description
unsigned long	pages_scanned	Counter used when doing page frame reclaiming in the zone.
Int	all_unreclaimable	Flag set when the zone is full of unreclaimable pages.
Int	temp_priority	Temporary zone's priority (used when doing page frame reclaiming).
Int	prev_priority	Zone's priority ranging between 12 and 0
wait_queue_head_t * wait_table		Hash table of wait queues of processes waiting for one of the pages of the zone.
unsigned long	wait_table_size	Size of the wait queue hash table.
unsigned long	wait_table_bits	Power-of-2 order of the size of the wait queue hash table array
struct pglist_data *	zone_pgdat	Memory node (NUMA)
struct page *	zone_mem_map	Pointer to first page descriptor of the zone.
unsigned long	zone_start_pfn	Index of the first page frame of the zone.
unsigned long	spanned_pages	Total size of zone in pages, including holes.
unsigned long	present_pages	Total size of zone in pages, excluding holes.
char *	name	Pointer to the conventional name of the zone: "DMA," "Normal," or "HighMem."

Page Frame Management



❑ Memory Zones

- Each page descriptor has links to the memory node and to the zone inside the node that includes the corresponding page frame.
- To save space, these links are not stored as classical pointers; rather, they are encoded as indices stored in the high bits of the flags field.
- In fact, the number of flags that characterize a page frame is limited, thus it is always possible to reserve the **most significant bits of the flags field** to encode the proper memory node and zone number.
- The **page_zone()** function receives as its parameter the address of a page descriptor; it reads the **most significant bits of the flags field** in the page descriptor, then it determines the address of the corresponding zone descriptor by looking in the **zone_table** array.
- This array is initialized at boot time with the addresses of all zone descriptors of all memory nodes.

Page Frame Management

- ❑ **Memory Zones – flags field bits to indicate node & zone**
 - The number of bits reserved for the indices depends on whether the kernel supports the NUMA model and on the size of the flags field.
 - If NUMA is not supported, the flags field has two bits for the zone index and one bit always set to zero for the node index.
 - On NUMA 32-bit architectures, flags has two bits for the zone index and six bits for the node number.
 - Finally, on NUMA 64-bit architectures, the 64-bit flags field has 2 bits for the zone index and 10 bits for the node number.
 - When the kernel invokes a memory allocation function, it must specify the zones that contain the requested page frames.
 - The kernel usually specifies which zones it's willing to use.
 - For instance, if a page frame must be directly mapped in the fourth gigabyte of linear addresses but it is not going to be used for ISA DMA transfers, then the kernel requests a page frame either in `ZONE_NORMAL` or in `ZONE_DMA`.
 - Of course, the page frame should be obtained from `ZONE_DMA` only if `ZONE_NORMAL` does not have free page frames.
 - To specify the preferred zones in a memory allocation request, the kernel uses **the zonelist** data structure, which is an array of zone descriptor pointers.

Page Frame Management

❑ Pool of Reserved Page Frames

- Memory allocation requests can be satisfied in two different ways.
- If enough free memory is available, the request can be satisfied immediately.
- Otherwise, some memory reclaiming must take place, and the kernel control path that made the request is blocked until additional memory has been freed.
- However, some kernel control paths cannot be blocked while requesting memory, this happens, for instance, when handling an interrupt or when executing code inside a critical region.
- In these cases, a kernel control path should issue atomic memory allocation requests (using the GFP_ATOMIC flag)
- An atomic request never blocks: if there are not enough free pages, the allocation simply fails.

Page Frame Management

❑ Pool of Reserved Page Frames

- Although there is no way to ensure that an atomic memory allocation request never fails, the kernel tries hard to minimize the likelihood of this unfortunate event.
- In order to do this, the kernel reserves a pool of page frames for atomic memory allocation requests to be used only on low-on-memory conditions.
- The amount of the reserved memory (in kilobytes) is stored in the `min_free_kbytes` variable. Its initial value is set during kernel initialization and depends on the amount of physical memory that is directly mapped in the kernel's fourth gigabyte of linear addresses that is, it depends on the number of page frames included in the `ZONE_DMA` and `ZONE_NORMAL` memory zones:

$$\text{reserved pool size} = \left\lfloor \sqrt{16 \times \text{directly mapped memory}} \right\rfloor \text{ (kilobytes)}$$

- However, initially `min_free_kbytes` cannot be lower than 128 and greater than 65,536.

Page Frame Management

□ Pool of Reserved Page Frames

- However, initially `min_free_kbytes` cannot be lower than 128 and greater than 65,536.

$$\text{reserved pool size} = \lfloor \sqrt{16 \times \text{directly mapped memory}} \rfloor \text{ (kilobytes)}$$

- The amount of reserved memory can be changed later by the system administrator either by writing in the `/proc/sys/vm/min_free_kbytes` file or by issuing a suitable `sysctl()` system call.
- The `ZONE_DMA` and `ZONE_NORMAL` memory zones contribute to the reserved memory with a number of page frames proportional to their relative sizes.
- For instance, if the `ZONE_NORMAL` zone is eight times bigger than `ZONE_DMA`, seven-eighths of the page frames will be taken from `ZONE_NORMAL` and one-eighth from `ZONE_DMA`.
- The `pages_min` field of the zone descriptor stores the number of reserved page frames inside the zone.
- This field plays also a role for the page frame reclaiming algorithm, together with the `pages_low` and `pages_high` fields.
- The `pages_low` field is always set to 5/4 of the value of `pages_min`, and `pages_high` is always set to 3/2 of the value of `pages_min`.

SOC 3010

OPERATING SYSTEM

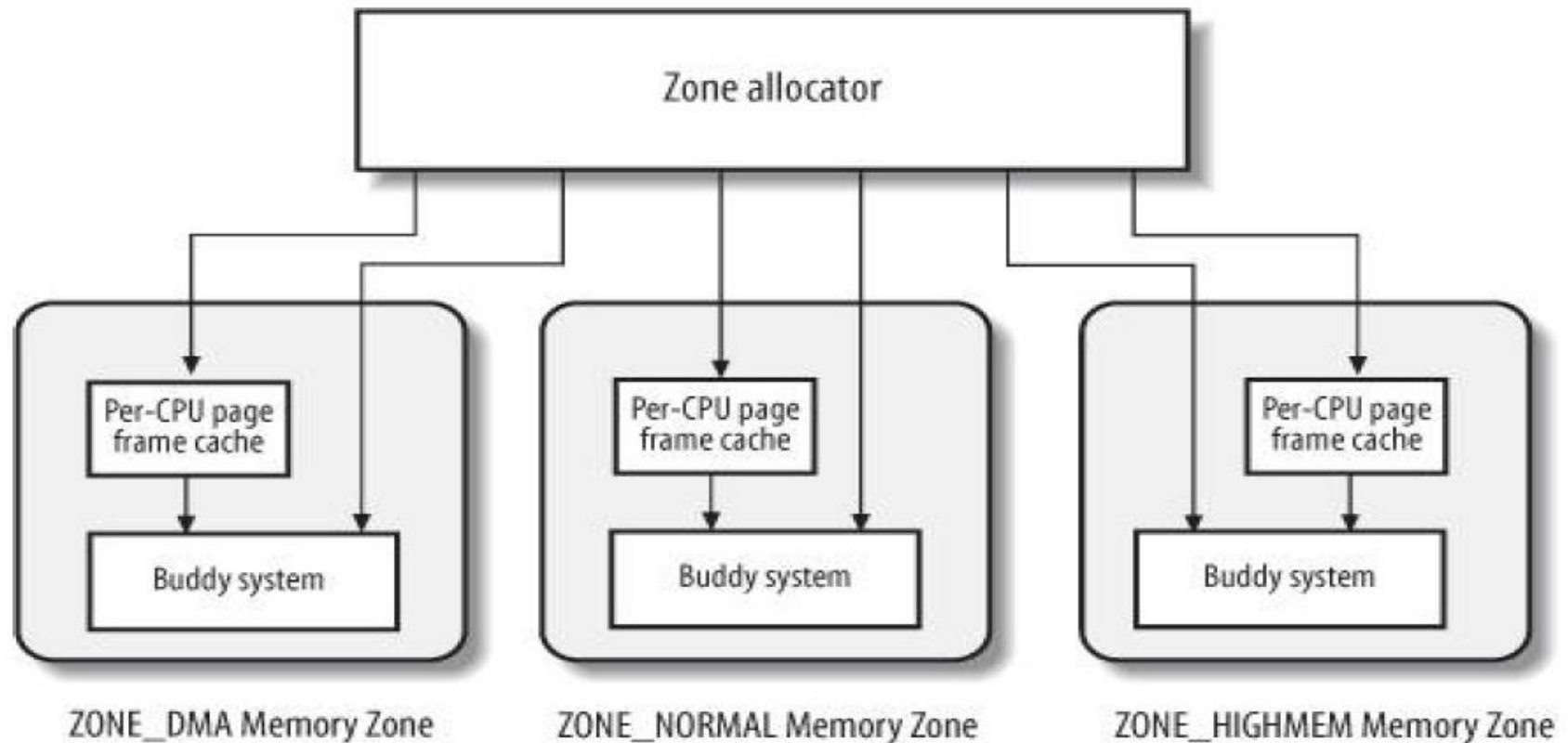
ZONED PAGE FRAME ALLOCATOR

Page Frame Management

- ❑ **Zoned Page Frame Allocator**
 - The kernel subsystem that handles the memory allocation requests for groups of contiguous page frames is called the zoned page frame allocator .

Page Frame Management

❑ Componets of a Zoned Page Frame Allocator



- The component named "zone allocator " receives the requests for allocation and deallocation of dynamic memory.
- In the case of allocation requests, the component searches a memory zone that includes a group of contiguous page frames that can satisfy the request.
- Inside each zone, page frames are handled by a component named "buddy system ".
- To get better system performance, a small number of page frames are kept in cache to quickly satisfy the allocation requests for single page frames .

Zoned Page Frame Allocator

❑ Requesting and releasing page frames

- Page frames can be requested by using six slightly different functions and macros. Unless otherwise stated, they return the linear address of the first allocated page or return NULL if the allocation failed.
- **alloc_pages(gfp_mask, order)** macro used to request 2^{order} contiguous page frames. It returns the address of the descriptor of the first allocated page frame or returns NULL if the allocation failed.
- **alloc_page(gfp_mask) Macro** used to get a single page frame; it expands to: `alloc_pages(gfp_mask, 0)`. It returns the address of the descriptor of the allocated page frame or returns NULL if the allocation failed.
- **__get_free_pages(gfp_mask, order) Function** that is similar to `alloc_pages()`, but it returns the linear address of the first allocated page.
- **get_free_page(gfp_mask) Macro** used to get a single page frame; it expands to: `__get_free_pages(gfp_mask, 0)`
- **__get_zeroed_page(gfp_mask) Function** used to obtain a page frame filled with zeros; it invokes: `alloc_pages(gfp_mask | __GFP_ZERO, 0)` and returns the linear address of the obtained page frame.
- **get_dma_pages(gfp_mask, order)** Macro used to get page frames suitable for DMA; it expands to: `__get_free_pages (gfp_mask | __GFP_DMA, order)`

Zoned Page Frame Allocator

❑ Requesting and releasing page frames

- The parameter `gfp_mask` is a group of flags that specify how to look for free page frames. The flags that can be used in `gfp_mask` are shown in Table

Table- Flag used to request page frames

Flag	Description
<code>__GFP_DMA</code>	The page frame must belong to the <code>ZONE_DMA</code> memory zone. Equivalent to <code>GFP_DMA</code> .
<code>__GFP_HIGHMEM</code>	The page frame may belong to the <code>ZONE_HIGHMEM</code> memory zone.
<code>__GFP_WAIT</code>	The kernel is allowed to block the current process waiting for free page frames.
<code>__GFP_HIGH</code>	The kernel is allowed to access the pool of reserved page frames.
<code>__GFP_IO</code>	The kernel is allowed to perform I/O transfers on low memory pages in order to free page frames.
<code>__GFP_FS</code>	If clear, the kernel is not allowed to perform filesystem-dependent operations.
<code>__GFP_COLD</code>	The requested page frames may be "cold"
<code>__GFP_NOWARN</code>	A memory allocation failure will not produce a warning message.
<code>__GFP_REPEAT</code>	The kernel keeps retrying the memory allocation until it succeeds.
<code>__GFP_NOFAIL</code>	Same as <code>__GFP_REPEAT</code> .
<code>__GFP_NORETRY</code>	Do not retry a failed memory allocation.
<code>__GFP_NO_GROW</code>	The slab allocator does not allow a slab cache to be enlarged
<code>__GFP_COMP</code>	The page frame belongs to an extended page
<code>__GFP_ZERO</code>	The page frame returned, if any, must be filled with zeros.

- In practice, Linux uses the predefined combinations of flag values shown in Table; the group name is what you'll encounter as the argument of the six page frame allocation functions.

Table - Groups of flag values used to request page frames

Group name	Corresponding flags
<code>GFP_ATOMIC</code>	<code>__GFP_HIGH</code>
<code>GFP_NOIO</code>	<code>__GFP_WAIT</code>
<code>GFP_NOFS</code>	<code>__GFP_WAIT</code> <code>__GFP_IO</code>
<code>GFP_KERNEL</code>	<code>__GFP_WAIT</code> <code>__GFP_IO</code> <code>__GFP_FS</code>
<code>GFP_USER</code>	<code>__GFP_WAIT</code> <code>__GFP_IO</code> <code>__GFP_FS</code>
<code>GFP_HIGHUSER</code>	<code>__GFP_WAIT</code> <code>__GFP_IO</code> <code>__GFP_FS</code> <code>__GFP_HIGHMEM</code>

Zoned Page Frame Allocator

- ❑ Page frames can be released through each of the following four functions and macros:
 - **__free_pages(page, order)** This function checks the page descriptor pointed to by page; if the page frame is not reserved (i.e., if the PG_reserved flag is equal to 0), it decreases the count field of the descriptor. If count becomes 0, it assumes that 2^{order} contiguous page frames starting from the one corresponding to page are no longer used. In this case, the function releases the page frames.
 - **__free_pages(addr, order)** This function is similar to **__free_pages()**, but it receives as an argument the linear address addr of the first page frame to be released.
 - **free_page(page)** This macro releases the page frame having the descriptor pointed to by page; it expands to: **__free_pages(page, 0)**
 - **free_page(addr)** This macro releases the page frame having the linear address addr; it expands to: **free_pages(addr, 0)**

Zoned Page Frame Allocator

❑ Kernel Mappings of High-Memory Page Frames

- The linear address that corresponds to the end of the directly mapped physical memory, and thus to the beginning of the high memory, is stored in the `high_memory` variable, which is set to 896 MB.
- Page frames above the 896 MB boundary are not generally mapped in the fourth gigabyte of the kernel linear address spaces, so the kernel is unable to directly access them.
- This implies that each page allocator function that returns the linear address of the assigned page frame doesn't work for high-memory page frames, that is, for page frames in the `ZONE_HIGHMEM` memory zone.
- For instance, suppose that the kernel invoked `__get_free_pages(GFP_HIGHMEM,0)` to allocate a page frame in high memory. If the allocator assigned a page frame in high memory, `__get_free_pages()` cannot return its linear address because it doesn't exist; thus, the function returns `NULL`. In turn, the kernel cannot use the page frame; even worse, the page frame cannot be released because the kernel has lost track of it.

Page Frame Management

❑ Kernel Mappings of High-Memory Page Frames

- The problem of non-allocation of high-memory page frames does not exist on 64-bit hardware platforms, because the available linear address space is much larger than the amount of RAM that can be installed in short, the `ZONE_HIGHMEM` zone of these architectures is always empty.
- On 32-bit platforms such as the 80x86 architecture, however, Linux designers had to find some way to allow the kernel to exploit all the available RAM, up to the 64 GB supported by PAE. The approach adopted is the following:
 - The allocation of high-memory page frames is done only through the `alloc_pages( )` function and its `alloc_page( )` shortcut.
 - These functions do not return the linear address of the first allocated page frame, because if the page frame belongs to the high memory, such linear address simply does not exist.
 - Instead, the functions return the linear address of the page descriptor of the first allocated page frame.
 - These linear addresses always exist, because all page descriptors are allocated in low memory once and forever during the kernel initialization.

Page Frame Management

❑ Kernel Mappings of High-Memory Page Frames

- Page frames in high memory that do not have a linear address cannot be accessed by the kernel.
- Therefore, part of the last 128 MB of the kernel linear address space is dedicated to mapping high-memory page frames.
- Of course, this kind of mapping is temporary, otherwise only 128 MB of high memory would be accessible.
- Instead, by recycling linear addresses the whole high memory can be accessed, although at different times.
- The kernel uses three different mechanisms to map page frames in high memory; they are called
 - Permanent kernel mapping,
 - Temporary kernel mapping, and
 - Noncontiguous memory allocation.

Page Frame Management

❑ Kernel Mappings of High-Memory Page Frames

- Establishing a permanent kernel mapping may block the current process; this happens when no free Page Table entries exist that can be used as "windows" on the page frames in high memory.
- Thus, a permanent kernel mapping cannot be established in interrupt handlers and deferrable functions.
- Conversely, establishing a temporary kernel mapping never requires blocking the current process; its drawback, however, is that very few temporary kernel mappings can be established at the same time.
- A kernel control path that uses a temporary kernel mapping must ensure that no other kernel control path is using the same mapping.
- This implies that the kernel control path can never block, otherwise another kernel control path might use the same window to map some other high memory page.
- Of course, none of these techniques allow addressing the whole RAM simultaneously.
- After all, less than 128 MB of linear address space are left for mapping the high memory, while PAE supports systems having up to 64 GB of RAM.

Page Frame Management

- ❑ Permanent kernel mappings
 - Permanent kernel mappings allow the kernel to establish long-lasting mappings of high-memory page frames into the kernel address space.
 - They use a dedicated Page Table in the master kernel page tables.
 - The `pkmap_page_table` variable stores the address of this Page Table, while the `LAST_PKMAP` macro yields the number of entries.
 - As usual, the Page Table includes either 512 or 1,024 entries, according to whether PAE is enabled or disabled, the kernel can access at most 2 or 4 MB of high memory at once.

Page Frame Management

❑ Permanent kernel mappings

- To keep track of the association between high memory page frames and linear addresses induced by permanent kernel mappings , the kernel makes use of the `page_address_htable` hash table.
- This table contains one `page_address_map` data structure for each page frame in high memory that is currently mapped.
- In turn, this data structure contains a pointer to the page descriptor and the linear address assigned to the page frame.
- The `page_address()` function returns the linear address associated with the page frame, or NULL if the page frame is in high memory and is not mapped.
- This function, which receives as its parameter a page descriptor pointer `page`, distinguishes two cases:
 - If the page frame is not in high memory (`PG_highmem` flag clear), the linear address always exists and is obtained by computing the page frame index, converting it into a physical address, and finally deriving the linear address corresponding to the physical address.
 - If the page frame is in high memory (`PG_highmem` flag set), the function looks into the `page_address_htable` hash table. If the page frame is found in the hash table, `page_address( )` returns its linear address, otherwise it returns NULL.

Page Frame Management

❑ Permanent kernel mappings

- The **kmap()** function establishes a permanent kernel mapping.
- The function gets **the kmap_lock spin lock** to protect the Page Table against concurrent accesses in multiprocessor systems.
- Notice that there is no need to disable the interrupts, because **kmap()** cannot be invoked by interrupt handlers and deferrable functions.
- The **kunmap()** function destroys a permanent kernel mapping established previously by **kmap()**.

Page Frame Management

- ❑ **Temporary kernel mappings**
 - **Temporary kernel mappings are simpler to implement than permanent kernel mappings; moreover, they can be used inside interrupt handlers and deferrable functions, because requesting a temporary kernel mapping never blocks the current process.**
 - **Every page frame in high memory can be mapped through a window in the kernel address space namely, a Page Table entry that is reserved for this purpose.**
 - **The number of windows reserved for temporary kernel mappings is quite small.**
 - **Each CPU has its own set of 13 windows, represented by the enum `km_type` data structure.**
 - **The kernel must ensure that the same window is never used by two kernel control paths at the same time.**

Zoned Page Frame Allocator

- ❑ Temporary kernel mappings
 - To establish a temporary kernel mapping, the kernel invokes the **kmap_atomic()** function
 - To destroy a temporary kernel mapping, the kernel uses the **kunmap_atomic()** function.
 - In the 80 x 86 architecture, this function decreases the preempt_count of the current process; thus, if the kernel control path was preemptable right before requiring a temporary kernel mapping, it will be preemptable again after it has destroyed the same mapping.
 - Moreover, **kunmap_atomic()** checks whether the TIF_NEED_RESCHED flag of current is set and, if so, invokes schedule().

SOC 3010

OPERATING SYSTEM

BUDDY SYSTEM ALGORITHM

Buddy System Algorithm

- The kernel must establish a robust and efficient strategy for allocating groups of contiguous page frames.
- In doing so, it must deal with a well-known memory management problem called external fragmentation: frequent requests and releases of groups of contiguous page frames of different sizes may lead to a situation in which several small blocks of free page frames are "scattered" inside blocks of allocated page frames.
- As a result, it may become impossible to allocate a large block of contiguous page frames, even if there are enough free pages to satisfy the request.
- There are essentially two ways to avoid external fragmentation:
 - Use the paging circuitry to map groups of non-contiguous free page frames into intervals of contiguous linear addresses.
 - Develop a suitable technique to keep track of the existing blocks of free contiguous page frames, avoiding as much as possible the need to split up a large free block to satisfy a request for a smaller one.

Buddy System Algorithm

- The second approach is preferred by the kernel for three good reasons:
- In some cases, contiguous page frames are really necessary, because contiguous linear addresses are not sufficient to satisfy the request.
- A typical example is a memory request for buffers to be assigned to a DMA processor.
- Because most DMAs ignore the paging circuitry and access the address bus directly while transferring several disk sectors in a single I/O operation, the buffers requested must be located in contiguous page frames.
- Even if contiguous page frame allocation is not strictly necessary, it offers the big advantage of leaving the kernel paging tables unchanged.
- **What's wrong with modifying the Page Tables?**
- Frequent Page Table modifications lead to higher average memory access times, because they make the CPU flush the contents of the translation lookaside buffers.
- Large chunks of contiguous physical memory can be accessed by the kernel through 4 MB pages. This reduces the translation lookaside buffers misses, thus significantly speeding up the average memory access time.

Buddy System Algorithm

- The technique adopted by Linux to solve the external fragmentation problem is based on the well-known buddy system algorithm.
- All free page frames are grouped into 11 lists of blocks that contain groups of 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, and 1024 contiguous page frames, respectively.
- The largest request of 1024 page frames corresponds to a chunk of 4 MB of contiguous RAM.
- The physical address of the first page frame of a block is a multiple of the group size for example, the initial address of a 16-page-frame block is a multiple of 16×2^{12} ($2^{12} = 4,096$, which is the regular page size).

Buddy System Algorithm

❑ Working of Buddy System Algorithm through a simple example:

- Assume there is a request for a group of 256 contiguous page frames (i.e., one megabyte).
- The algorithm checks first to see whether a free block in the 256-page-frame list exists.
- If there is no such block, the algorithm looks for the next larger block, a free block in the 512-page-frame list.
- If such a block exists, the kernel allocates 256 of the 512 page frames to satisfy the request and inserts the remaining 256 page frames into the list of free 256-page-frame blocks.
- If there is no free 512-page block, the kernel then looks for the next larger block (i.e., a free 1024-page-frame block).
- If such a block exists, it allocates 256 of the 1024 page frames to satisfy the request, inserts the first 512 of the remaining 768 page frames into the list of free 512-page-frame blocks, and inserts the last 256 page frames into the list of free 256-page-frame blocks.
- If the list of 1024-page-frame blocks is empty, the algorithm gives up and signals an error condition.

Buddy System Algorithm

- ❑ Working of Buddy System Algorithm through a simple example:
 - ★ The reverse operation, releasing blocks of page frames, gives rise to the name of this algorithm.
 - ★ The kernel attempts to merge pairs of free buddy blocks of size b together into a single block of size $2b$.
 - ★ Two blocks are considered buddies if:
 - Both blocks have the same size, say b .
 - They are located in contiguous physical addresses.
 - The physical address of the first page frame of the first block is a multiple of $2 \times b \times 2^{12}$.
 - The algorithm is iterative; if it succeeds in merging released blocks, it doubles b and tries again so as to create even bigger blocks.

Buddy System Algorithm

❑ Data structures

- Linux uses a different buddy system for each zone.
- Thus, in the 80 x 86 architecture, there are 3 buddy systems: the first handles the page frames suitable for ISA DMA, the second handles the "normal" page frames, and the third handles the high-memory page frames.
- Each buddy system relies on the following main data structures :
- The **mem_map array** - Actually, each zone is concerned with a subset of the mem_map elements.
- The first element in the subset and its number of elements are specified, respectively, by the **zone_mem_map** and **size** fields of the zone descriptor.
- An array consisting of eleven elements of type free_area, one element for each group size.
- The array is stored in the **free_area** field of the zone descriptor.

Buddy System Algorithm

❑ Allocating a block

- The **__rmqueue()** function is used to find a free block in a zone.
- The function takes two arguments: the address of the zone descriptor, and order, which denotes the logarithm of the size of the requested block of free pages (0 for a one-page block, 1 for a two-page block, and so forth).
- If the page frames are successfully allocated, the **__rmqueue()** function returns the address of the page descriptor of the first allocated page frame. Otherwise, the function returns NULL.
- The **__rmqueue()** function assumes that the caller has already disabled local interrupts and acquired the zone->lock spin lock, which protects the data structures of the buddy system.

Buddy System Algorithm

❑ Freeing a block

- The **__free_pages_bulk()** function implements the buddy system strategy for freeing page frames.
- It uses three basic input parameters:
- For performance reasons, this inline function also uses another parameter; its value, however, can be determined by the three basic parameters :
 - Page** - The address of the descriptor of the first page frame included in the block to be released
 - zone** - The address of the zone descriptor
 - order** - The logarithmic size of the block
- The function assumes that the caller has already disabled local interrupts and acquired the zone->lock spin lock, which protects the data structure of the buddy system.

Buddy System Algorithm



❑ The Per-CPU Page Frame Cache

- ★ The kernel often requests and releases single page frames.
- ★ To boost system performance, **each memory zone defines a per-CPU page frame cache.**
- ★ **Each per-CPU cache includes some pre-allocated page frames to be used for single memory requests issued by the local CPU.**
- ★ **Actually, there are two caches for each memory zone and for each CPU:**
 - **a hot cache** - which stores page frames whose contents are likely to be included in the CPU's hardware cache,
 - **a cold cache** - which stores page frames for DMA operation

Buddy System Algorithm



❑ The Per-CPU Page Frame Cache

➤ Hot Cache

- ★ Taking a page frame from the hot cache is beneficial for system performance if either the kernel or a User Mode process will write into the page frame right after the allocation.
- ★ In fact, every access to a memory cell of the page frame will result in a line of the hardware cache being "stolen" from another page frame unless, of course, the hardware cache already includes a line that maps the cell of the "hot" page frame just accessed.

➤ Cold Cache

- ★ Conversely, taking a page frame from the cold cache is convenient if the page frame is going to be filled with a DMA operation.
- ★ In this case, the CPU is not involved and no line of the hardware cache will be modified.
- ★ Taking the page frame from the cold cache preserves the reserve of hot page frames for the other kinds of memory allocation requests.

Buddy System Algorithm

- ❑ **Allocating page frames through the per-CPU page frame caches**
 - The **buffered_rmqueue()** function allocates page frames in a given memory zone.
 - It makes use of the per-CPU page frame caches to handle single page frame requests.
 - The parameters are the address of the memory zone descriptor, the order of the memory allocation request order, and the allocation flags **gfp_flags**.

- ❑ **Releasing page frames to the per-CPU page frame caches**
 - In order to release a single page frame to a per-CPU page frame cache, the kernel makes use of the **free_hot_page()** and **free_cold_page()** functions.
 - Both of them are simple wrappers for the **free_hot_cold_page()** function, which receives as its parameters the descriptor address page of the page frame to be released and a cold flag specifying either the hot cache or the cold cache.

MEMORY MANAGEMENT

Questions

- * Explain the following:
 - a) Memory Management
 - b) Virtual memory
 - c) Protected memory
 - d) Shared memory
- * Explain why the kernel must keep track of the current status of each page frame.
- * Explain the purpose of page descriptors.
- * State information of a page frame is kept in a page descriptor of type
- * All page descriptors are stored in the array.
- * Size of each page descriptor is bytes
 - a) 8
 - b) 16
 - c) 32
 - d) 64
- * Given a main memory of 4GB, determine the maximum number of page frames in this memory and the size of the mem_map array required to store all the page descriptors.
- * The macro yields the address of the page descriptor associated with the linear address addr.
- * The macro yields the address of the page descriptor associated with the page frame having number pfn.
- * State the purpose of the following macros :
 - a) virt_to_page(addr)
 - b) pfn_to_page(pfn)
- * Describe the purpose of the following fields in the page descriptor :
 - a) flags
 - b) _count
 - c) _mapcount
 - d) private
 - e) mapping
 - f) index
 - g) lru
- * Explain the following :
 - a) Uniform Memory Access Architecture (UMA)
 - b) Non-Uniform Memory Access Architecture (NUMA)
- * Indicate the number of memory nodes in UMA architecture.
- * Indicate the maximum number of memory nodes in NUMA architectures considering both 32-bit and 64-bit architectures.
- * Maximum number of memory nodes in 32-bit architecture is and number of bits required in flags field to store both zone and node numbers is
- * Maximum number of memory nodes in 64-bit architecture is and number of bits required in flags field to store both zone and node numbers is
- * Discuss the two hardware constraints of the 80x86 32-bit architecture that may limit the way page frames can be used.
- * Explain how Linux Partitions the physical memory of every memory node into Memory zones. Describe these Memory zones

MEMORY MANAGEMENT

□ Questions

- * Which field bits in the page descriptor are used to specify the memory node number and the zone number? How many bits are used to specify memory node number and the zone number in UMA and NUMA architectures?
- * Explain the following :
 - a) DMA Zone
 - b) NORMAL Zone
 - c) HIGH MEMORY Zone
- * Size of DMA Zone isMB and size of NORMAL Zone is MB
- * Explain what is meant by Page reclaiming. Indicate when is it required and which kernel thread is invoked to perform page reclaiming.
- * State the purpose of the kernel thread kswapd.
- * Each node has a descriptor of type and all node descriptors are stored in a singly linked list, whose first element is pointed to by the variable.
- * Explain the purpose of the following fields in memory zone descriptor :
 - a) Free_pages
 - b) pages_min
 - c) pages_low
 - d) pages_high
 - e) lowmem_reserve
- * State the purpose of the function page_zone().
- * What are Pool of Reserved Page Frames? Why are they required? Where they are located?
- * Explain the two kernel control paths which make use of atomic memory allocation requests. State the need for having these separate memory allocation requests.
- * Write the expression for computing reserved pool size. What are the minimum and maximum values of the reserved pool size that can be stored in the variable min_free_kbytes? Also specify the directly mapped memory sizes used to compute these values.
- * Describe the approach used to build the pool of Reserved Page frames by considering the sizes of DMA Zone and NORMAL Zone.
- * State the purpose of pages_min field in the memory zone descriptor. Show how this field is used to compute values of pages_low and pages_high fields in the zone descriptor?
- * With the help of a schematic, describe the Zoned Page Frame Allocator. Explain the purpose of all the components in the system.

MEMORY MANAGEMENT

□ Questions

- * Describe the following components in the Zoned Page Frame Allocator:
 - a) Zone Allocator
 - b) Buddy System
 - c) Per-CPU Page Frame Cache
- * Describe the SIX different functions used to request page frames allocation.
- * What is gfp_mask parameter in allocation function? Why is it required?
- * State the purpose of __GFP_DMA and __GFP_HIGHMEM flags of the gfp_mask.
- * Describe the FOUR different functions used to release page frames.
- * Describe the approach adopted on 32-bit architectures to exploit all the available RAM, up to the 64 GB supported by PAE.
- * Discuss the three different mechanisms used by the kernel to map page frames in high memory.
- * Describe the following mechanisms used by the kernel to map page frames in high memory clearly providing the approach used.
 - a) Permanent Kernel Mapping
 - b) Temporary Kernel mapping
 - c) Non-contiguous Memory Allocation
- * What are the two major problems of Memory Management tackled by Buddy algorithm?
- * Explain the following memory management problems :
 - a) External Fragmentation
 - b) Internal Fragmentation
- * Describe the working of a Buddy System Algorithm through a simple example and state how this algorithm gets the name Buddy
- * What are the functions used to allocate and deallocate a block in a zone?
- * State the purpose of Per_CPU Page frame cache.
- * Describe the two per-CPU Page frame caches defined for each memory zone.
- * Explain the following Per_CPU page frame caches:
 - a) Hot cache
 - b) Cold cache
- * Write the functions used to allocate and release page frames from the Per_CPU page frame cache.

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 8 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010
OPERATING SYSTEM

END OF
WEEK 13 LECTURE2

MEMORY MANAGEMENT –
PART1

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

